



e

E-Market

DOCUMENTAÇÃO TÉCNICA COMPLETA

E-Market

Arquitetura, implementação, segurança,
banco de dados, APIs e operação

Referência completa da implementação full stack localizada em
C:\Users\erick\Desktop\e-market, incluindo frontend Next.js, backend NestJS e PostgreSQL.

Versão 1.0 | 09 de agosto de 2026

Sobre este documento

Este manual descreve o estado técnico atual do E-Market e serve como referência para desenvolvimento, manutenção, demonstração de portfólio, testes e futura implantação. As informações foram consolidadas a partir do código-fonte, do schema Prisma, das migrations e dos scripts de validação presentes no projeto.

Privacidade: Credenciais reais, conteúdo de arquivos .env, senhas, tokens e dados pessoais cadastrados no PostgreSQL não fazem parte desta documentação.

Conteúdo

1. Visão geral e escopo funcional
2. Arquitetura e fluxo de comunicação
3. Estrutura do repositório e tecnologias
4. Frontend, experiência e gerenciamento de estado
5. Backend, módulos e regras de negócio
6. Autenticação, sessões e segurança
7. Modelo de dados e integridade no PostgreSQL
8. Referência das APIs
9. Fluxos ponta a ponta
10. Configuração, execução local e banco
11. Testes, diagnóstico e operação
12. Implantação, limitações e evolução recomendada

Estado da solução

Área	Situação atual
Autenticação	JWT, bcrypt, cookie HttpOnly, login por e-mail ou CPF, cadastro e recuperação local demonstrativa.
Catálogo	Produtos e categorias carregados pela API, pesquisa, ofertas e página de detalhes.
Compra	Carrinho persistido, favoritos, endereço, cupom, checkout e pedido transacional.
Pedidos	Histórico, expiração, confirmação demonstrativa, cancelamento, estoque e estorno lógico.
Conta	Perfil, telefone, e-mail, endereços, preferências e tema claro/escuro persistidos.
Administração	Dashboard, clientes, pedidos, status, produtos e ajustes de estoque via API.

1. Visão geral do sistema

O E-Market é uma aplicação full stack de supermercado online voltada a portfólio. O sistema exige autenticação para acessar as páginas comerciais, separa responsabilidades entre interface, camada BFF e API REST, e persiste os dados no PostgreSQL por meio do Prisma ORM.

1.1 Objetivos funcionais

- Permitir cadastro de cliente com validação de CPF, telefone, endereço e senha forte.
- Autenticar por e-mail ou CPF e manter a sessão sem expor o token ao JavaScript do navegador.
- Exibir catálogo organizado por departamentos, busca, ofertas e detalhes de produto.

- Persistir carrinho, favoritos, preferências, endereços, pedidos e movimentações de estoque.
- Executar checkout com recálculo de preços, validação de estoque e cupom dentro de transação.
- Disponibilizar recursos administrativos protegidos por função de usuário.

1.2 Perfis de acesso

Perfil	Permissões
CLIENT	Catálogo, conta, endereços, carrinho, favoritos, checkout, pedidos e preferências.
EMPLOYEE	Rotas administrativas operacionais: pedidos, produtos e estoque.
ADMIN	Todos os recursos administrativos, incluindo ativação e bloqueio de clientes.

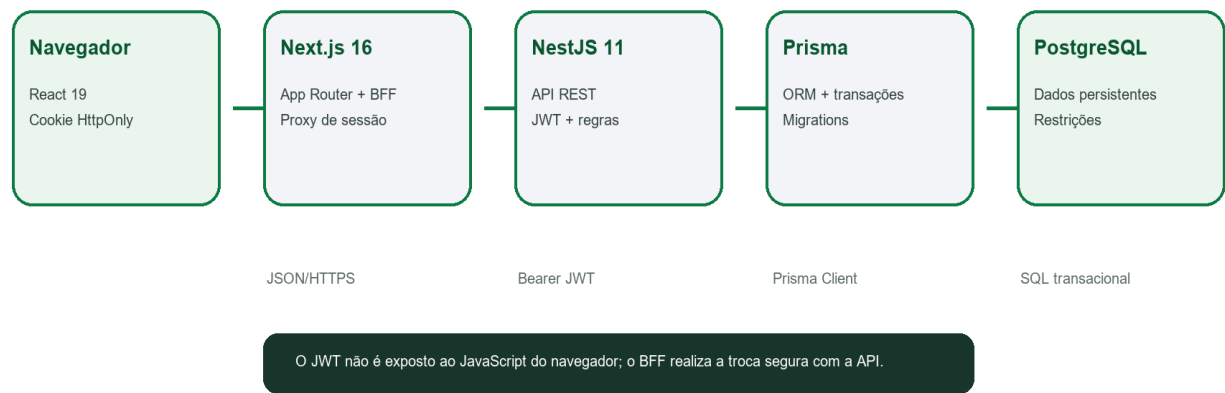
1.3 Limites do escopo atual

A integração de pagamento existente é demonstrativa e não movimenta dinheiro. Login social, envio real de e-mail, emissão de nota fiscal e integração logística externa ainda dependem de provedores e credenciais próprias. Essas ausências são tratadas como extensões de produto, não como funcionalidades simuladas silenciosamente.

2. Arquitetura

A arquitetura utiliza o padrão Backend for Frontend (BFF). O navegador acessa rotas do Next.js; operações autenticadas são encaminhadas pelo servidor Next.js ao NestJS, que aplica autorização e regras de negócio antes de consultar o PostgreSQL.

Fluxo técnico principal



2.1 Responsabilidades por camada

Camada	Responsabilidade principal
Navegador / React	Renderização, navegação, formulários, animações, estado otimista e feedback visual.
Next.js / BFF	Sessão em cookie HttpOnly, proteção de origem, proxy autenticado, redirecionamentos e SSR.
NestJS	Validação, autenticação JWT, autorização por roles, regras de negócio e rate limiting.

Camada	Responsabilidade principal
Prisma	Acesso tipado ao banco, transações, migrations e operações atômicas.
PostgreSQL	Persistência relacional, índices, unicidade e constraints de integridade.

2.2 Fronteiras de confiança

- O navegador nunca recebe o JWT por código cliente; o token fica no cookie emarket-session com HttpOnly.
- O BFF converte o cookie em Authorization: Bearer apenas na comunicação servidor a servidor.
- A API não confia em IDs enviados pelo cliente para determinar propriedade; usa o sub do JWT nas consultas.
- Operações de mudança no BFF exigem origem confiável e bloqueiam solicitações externas.
- O PostgreSQL reforça regras críticas mesmo se uma falha futura ultrapassar a validação da aplicação.

3. Estrutura e tecnologias

3.1 Organização do repositório

```
e-market/
├── backend/
│   ├── prisma/ schema, migrations e seed
│   ├── src/ módulos NestJS e regras de negócio
│   └── test/ testes E2E e smoke tests
├── frontend/
│   ├── app/ páginas, BFF, componentes e estilos
│   ├── public/ ativos públicos
│   └── proxy.ts proteção das rotas do app
├── docs/ documentação técnica
└── README.md instruções rápidas do projeto
```

3.2 Stack

Área	Tecnologias
Frontend	Next.js 16.3.0, React 19.2.4, TypeScript, App Router, Turbopack e CSS próprio.
Backend	NestJS 11, TypeScript, Class Validator, JWT, bcrypt, Helmet e Throttler.
Dados	PostgreSQL, Prisma Client/CLI 6.19.3 e migrations SQL.
Qualidade	ESLint, Prettier, Jest, Supertest, TypeScript estrito e scripts smoke.

3.3 Decisões técnicas relevantes

- CSS e ícones SVG próprios reduzem dependências visuais e mantêm a identidade do projeto.
- Prisma Decimal é utilizado nos cálculos monetários do backend para evitar aritmética binária imprecisa.
- O carrinho usa atualização otimista no frontend, seguida de reconciliação com a resposta do servidor.
- Pedidos usam transação e operações condicionais updateMany para reduzir conflitos concorrentes.
- O catálogo público da API é somente leitura; páginas do aplicativo continuam protegidas pelo proxy do Next.js.

4. Frontend

O frontend está em `frontend/app` e utiliza App Router. Páginas comerciais são envolvidas pelo componente Shell, que fornece cabeçalho, busca, sidebar persistente, tema, favoritos, carrinho e rodapé. As telas públicas de autenticação utilizam AuthFrame e não exibem a navegação do mercado.

4.1 Rotas de interface

Rota	Finalidade	Acesso
/	Entrada e carregamento inicial	Pública
/login	Login e confirmação animada	Pública
/cadastro	Criação de conta e endereço inicial	Pública
/recuperar-senha	Solicitação e redefinição de senha	Pública
/mercado	Página principal e ofertas	Protegida
/categorias/[slug]	Produtos por departamento	Protegida
/produtos/[slug]	Detalhes do produto	Protegida
/pesquisa	Busca e ordenação	Protegida
/ofertas	Produtos promocionais	Protegida
/favoritos	Itens salvos	Protegida
/carrinho	Itens, quantidades e subtotal	Protegida
/checkout	Endereço, entrega, cupom e pagamento	Protegida
/pedidos	Histórico e ações de pedido	Protegida
/perfil	Dados pessoais e endereços	Protegida
/configuracoes	Tema e comunicações	Protegida

4.2 Estado global

StoreProvider centraliza carrinho, favoritos, notificações e tema. A carga inicial autenticada consulta `/cart`, `/favorites` e `/auth/me` em paralelo. Operações por item possuem trava local para evitar cliques duplicados enquanto uma requisição está pendente.

- Carrinho: lista, quantidade total, valor total, adicionar, alterar, remover, limpar e recarregar.
- Favoritos: atualização otimista com restauração em caso de erro.
- Tema: preferência local imediata e persistência posterior na conta.
- Erros: timeout de 12 segundos e mensagem amigável; respostas 401 redirecionam ao login.

4.3 BFF e proteção de rotas

As rotas internas `/api/session`, `/api/logout`, `/api/profile` e `/api/backend/[...path]` mantêm a sessão fora do bundle cliente. O proxy.ts valida a sessão no backend antes de liberar páginas protegidas. Em indisponibilidade temporária da API, o acesso falha de forma fechada sem apagar imediatamente um cookie potencialmente válido.

4.4 Tema e responsividade

O tema é aplicado por data-theme no elemento html. Um script inicial mínimo lê a preferência local antes da hidratação para evitar mudança visual tardia. O layout usa sidebar recolhível, ocupa a largura disponível e adapta navegação e conteúdo para telas menores.

5. Backend e regras de negócio

O backend segue a organização modular do NestJS. ValidationPipe global remove campos não declarados, rejeita propriedades extras, transforma tipos e interrompe no primeiro erro. O ThrottlerGuard é global e o Helmet adiciona cabeçalhos defensivos à API.

5.1 Módulos

Módulo	Responsabilidade
Auth	Cadastro, login, sessão, perfil, configurações, recuperação e logout.
Products	Catálogo, categorias, busca, filtros, ofertas e detalhes.
Cart	Carrinho ativo, itens, quantidades e preços de referência.
Favorites	Listagem e alternância de favoritos por cliente.
Addresses	CRUD, endereço principal e limite de registros.
Coupons	Validação de vigência, uso, pedido mínimo e desconto.
Orders	Checkout transacional, pagamento demo, expiração e cancelamento.
Admin	Dashboard, clientes, pedidos, produtos e estoque.
Prisma	Conexão compartilhada e acesso ao PostgreSQL.

5.2 Catálogo e preços

A listagem limita o retorno a 100 produtos ativos e aceita categoria, busca, ordenação, preço mínimo/máximo e oferta. Filtros monetários consideram o preço promocional quando ele existe. O backend rejeita intervalo cujo mínimo seja maior que o máximo.

5.3 Carrinho e favoritos

Cada usuário possui no máximo um carrinho ACTIVE, garantido por índice parcial no banco. O serviço valida disponibilidade e estoque antes de adicionar ou alterar. Favoritos usam chave única composta por usuário e produto.

5.4 Checkout e pedidos

1. Carregar o carrinho ativo e confirmar que o endereço pertence ao usuário.
2. Recalcular preços usando os dados atuais do produto, sem confiar no subtotal do navegador.
3. Calcular frete e validar o cupom vigente, limite de uso e pedido mínimo.
4. Reivindicar o carrinho como CONVERTED para impedir checkout duplicado.
5. Baixar estoque com condição stock >= quantity para cada item.
6. Incrementar o uso do cupom de forma condicional.
7. Criar pedido, itens, pagamento pendente e movimentações de estoque na mesma transação.

Atomicidade: Se qualquer etapa falhar, a transação é revertida; estoque, cupom, carrinho e pedido não ficam parcialmente atualizados.

5.5 Expiração e cancelamento

Pedidos aguardando pagamento expiram após 30 minutos. Uma rotina executada a cada minuto reivindica até 100 pedidos vencidos, devolve estoque, devolve o uso do cupom e marca o pagamento como FAILED. Confirmações demonstrativas e alterações administrativas também verificam o prazo.

O cliente pode cancelar pedidos AWAITING_PAYMENT ou PAID. O estoque é restaurado e o pagamento lógico passa a FAILED ou REFUNDED. Em produção real, essa etapa deverá acionar a API do provedor e processar webhooks idempotentes.

6. Autenticação e segurança

6.1 Cadastro e senha

- Senha entre 8 e 64 caracteres, com maiúscula, minúscula, número e símbolo.
- Hash bcrypt com custo 12; a senha em texto puro não é persistida.
- E-mail é normalizado em minúsculas; CPF, telefone e CEP mantêm apenas dígitos.
- CPF é validado pelos dígitos verificadores e sequências repetidas são rejeitadas.
- E-mail, CPF e telefone possuem unicidade no PostgreSQL.

6.2 JWT e sessão

Controle	Implementação
Algoritmo	HS256 com secret de no mínimo 32 caracteres e rejeição do valor de exemplo.
Claims	sub, email, name, role e version; issuer e audience são verificados.
Validade	1 dia.
Cookie	HttpOnly, SameSite=Lax, Secure em produção, path=/ e maxAge de 1 dia.
Revogação	sessionVersion é comparada a cada rota protegida; logout e troca de senha incrementam a versão.

6.3 Recuperação de senha

O token é aleatório, possui 32 bytes representados em hexadecimal e somente seu SHA-256 é salvo. O prazo é de 30 minutos. Um novo pedido remove tokens anteriores do usuário; após a troca, tokens remanescentes são excluídos e todas as sessões são revogadas.

Modo local: O token só pode retornar na resposta quando NODE_ENV não é production e PASSWORD_RESET_MODE=demo. Fora do desenvolvimento, a configuração recomendada é disabled e o token deve ser enviado por e-mail.

6.4 Controles HTTP

- CORS com lista explícita de origens e credenciais habilitadas somente para essas origens.
- Proteção CSRF no BFF por comparação de Origin/Host e Sec-Fetch-Site para requisições sem Origin.
- CSP, frame-ancestors none, X-Frame-Options DENY, nosniff, Referrer-Policy e Permissions-Policy.
- HSTS e upgrade-insecure-requests em produção.
- Body máximo de 64 KiB no proxy autenticado e timeout de 12 segundos.
- Rate limit global de 120 requisições/minuto; login e cadastro 5/minuto; recuperação 3/minuto.

6.5 Autorização e exposição de dados

JwtAuthGuard consulta o usuário atual para validar atividade, versão de sessão e role. RolesGuard protege o namespace admin. Consultas de cliente incluem userId derivado do token, evitando acesso horizontal a carrinhos, endereços e pedidos de outra conta. Respostas de perfil e administração usam select explícito e não retornam password.

7. Banco de dados

O schema utiliza PostgreSQL e Prisma. Chaves estrangeiras representam propriedade e histórico. Exclusões usam Cascade quando os dados dependem integralmente do pai, Restrict quando o histórico precisa ser preservado e SetNull quando a referência pode deixar de existir sem apagar o registro histórico.

7.1 Modelos

Modelo	Finalidade
User	Conta, credenciais, role, estado, preferências e versão da sessão.
PasswordResetToken	Hash, validade, consumo e vínculo com usuário.
Address	Endereços do usuário e marcador principal.
Category	Departamento ativo identificado por slug.
Product	Produto, marca, código, unidade, preços, estoque e categoria.
ProductImage	Imagens ordenadas relacionadas ao produto.
Favorite	Associação única entre usuário e produto.
Cart / CartItem	Carrinho por estado e itens com quantidade/preço de referência.
Coupon	Regra de desconto, vigência, limite e contador de uso.
Order / OrderItem	Snapshot financeiro, entrega, status e itens históricos.
Payment	Método, status, referência externa, valor e comprovante.
StockMovement	Auditoria de entrada, saída e ajuste de estoque.

7.2 Constraints e índices críticos

- Um único carrinho ACTIVE por usuário e um único endereço principal por usuário.
- Um único token de recuperação não utilizado por usuário.
- Estoque, preços, pagamentos, cupons, quantidades e totais não podem assumir estados inválidos.
- Order.total deve ser exatamente subtotal + shippingFee - discount.
- OrderItem.subtotal deve ser unitPrice × quantity.
- Índices por usuário, categoria, produto, pedido, endereço e expiração suportam as consultas principais.

7.3 Enumerações

Enum	Valores
UserRole	CLIENT, ADMIN, EMPLOYEE
CartStatus	ACTIVE, CONVERTED, ABANDONED

Enum	Valores
CouponType	PERCENTAGE, FIXED
OrderStatus	AWAITING_PAYMENT, PAID, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELED, EXPIRED
ShippingMethod	DELIVERY, PICKUP
PaymentMethod	PIX, CREDIT_CARD, DEBIT_CARD, BOLETO
PaymentStatus	PENDING, PAID, FAILED, REFUNDED
StockMovementType	IN, OUT, ADJUSTMENT

8. Referência da API

Base local: <http://127.0.0.1:3000>. Rotas marcadas como JWT exigem Authorization: Bearer no backend; pelo frontend, o BFF injeta esse cabeçalho com base no cookie HttpOnly.

Método	Rota	Acesso	Finalidade
GET	/	Pública	Saúde da API.
POST	/auth/register	Pública	Cadastrar usuário e emitir sessão.
POST	/auth/login	Pública	Entrar por e-mail ou CPF.
POST	/auth/forgot-password	Pública	Criar recuperação sem revelar cadastro.
POST	/auth/reset-password	Pública	Trocar senha com token válido.
GET	/auth/me	JWT	Obter conta e preferências.
PATCH	/auth/me	JWT	Atualizar nome, e-mail e telefone.
PATCH	/auth/settings	JWT	Atualizar tema e comunicações.
POST	/auth/logout	JWT	Revogar sessões atuais.
GET	/products	Pública	Listar e filtrar produtos.
GET	/products/categories	Pública	Listar categorias ativas.
GET	/products/:slug	Pública	Detalhar produto ativo.
GET	/cart	JWT	Obter ou criar carrinho ativo.
POST	/cart/items	JWT	Adicionar item.
PATCH	/cart/items/:productId	JWT	Alterar quantidade.
DELETE	/cart/items/:productId	JWT	Remover item.
DELETE	/cart/items	JWT	Limpar carrinho.
GET	/favorites	JWT	Listar favoritos.
POST	/favorites/toggle	JWT	Adicionar/remover favorito.
GET	/addresses	JWT	Listar endereços.
POST	/addresses	JWT	Criar endereço.

Método	Rota	Acesso	Finalidade
PATCH	/addresses/:id	JWT	Editar endereço próprio.
DELETE	/addresses/:id	JWT	Remover endereço próprio.
GET	/coupons/validate	JWT	Pré-validar cupom.
GET	/orders	JWT	Listar pedidos próprios.
GET	/orders/:id	JWT	Detalhar pedido próprio.
POST	/orders	JWT	Finalizar checkout.
PATCH	/orders/:id/cancel	JWT	Cancelar quando permitido.
PATCH	/orders/:id/payment/confirm-demo	JWT	Confirmar pagamento demo.
GET	/admin/dashboard	Equipe	Métricas administrativas.
GET	/admin/orders	Equipe	Listar todos os pedidos.
PATCH	/admin/orders/:id/status	Equipe	Avançar status permitido.
GET	/admin/clients	Equipe	Listar clientes.
PATCH	/admin/clients/:id/status	Admin	Ativar/bloquear cliente.
POST	/admin/stock/:productId	Equipe	Ajustar estoque.
POST	/admin/products	Equipe	Criar produto.
PATCH	/admin/products/:id	Equipe	Editar produto.

8.1 Parâmetros do catálogo

Parâmetro	Descrição
category	Slug da categoria.
q	Trecho de nome ou marca, limitado internamente.
sort	price-asc, price-desc, name ou mais recente.
minPrice / maxPrice	Intervalo não negativo aplicado ao preço vigente.
offer=true	Somente produtos com preço promocional.

9. Fluxos ponta a ponta

9.1 Cadastro e login

1. O formulário envia os dados para /api/session no Next.js.
2. O BFF valida a origem e encaminha o payload ao NestJS.
3. O backend normaliza os campos, valida CPF/senha e cria usuário + endereço principal.
4. bcrypt gera o hash; o JWT recebe issuer, audience, role e sessionVersion.
5. O BFF grava o token em cookie HttpOnly e direciona à animação de sucesso.
6. O proxy valida /auth/me antes de liberar /mercado.

9.2 Compra

1. Cliente navega no catálogo e adiciona produtos ao carrinho persistido.
2. Checkout carrega endereço e recalcula subtotal a partir do estado sincronizado.
3. Cupom é pré-validado para feedback, mas validado novamente no backend.
4. POST /orders executa a transação de estoque, cupom, pedido, itens e pagamento.
5. Pagamento demo pode ser confirmado no checkout ou retomado em Meus pedidos antes do vencimento.
6. Pedido aparece no histórico com status e ações permitidas.

9.3 Cancelamento e expiração

Tanto o cancelamento quanto a expiração utilizam reivindicação condicional do status. A operação vencedora restaura estoque e cupom uma única vez; operações concorrentes recebem erro de pedido já atualizado.

9.4 Tema e conta

O tema muda imediatamente no cliente, é salvo em localStorage para carregamento sem flash e persistido em /auth/settings. Perfil e endereços usam rotas autenticadas e retornam ao usuário apenas seus próprios registros.

10. Configuração e execução local

10.1 Requisitos

- Node.js 20 ou superior.
- npm.
- PostgreSQL acessível.
- VS Code ou terminal PowerShell.

10.2 Variáveis do backend

Variável	Finalidade	Recomendação
DATABASE_URL	Conexão PostgreSQL.	Obrigatória; usar TLS em produção.
JWT_SECRET	Assinatura HS256.	Aleatória, exclusiva e ≥ 32 caracteres.
CORS_ORIGINS	Origens permitidas.	Lista explícita separada por vírgulas.
PORT	Porta da API.	3000 localmente.
PAYMENT_MODE	Libera pagamento demo em produção.	disabled por padrão.
PASSWORD_RESET_MODE	Exposição local do token.	disabled fora do desenvolvimento.

10.3 Variável do frontend

Variável	Finalidade
API_URL	URL interna do backend usada apenas pelo servidor Next.js.

10.4 Preparação do banco

```
cd C:\Users\erick\Desktop\e-market\backend
Copy-Item .env.example .env
npm install
npm run db:generate
npm run db:migrate
npm run db:seed
```

10.5 Inicialização em dois terminais

```
# Terminal 1 - API
cd C:\Users\erick\Desktop\e-market\backend
npm run start:dev

# Terminal 2 - interface
cd C:\Users\erick\Desktop\e-market\frontend
npm run dev
```

A interface fica em <http://127.0.0.1:3001> e a API em <http://127.0.0.1:3000>.

10.6 Migrations

Em ambientes existentes, use `npm run db:migrate`. O comando aplica somente migrations ainda não registradas no banco. Não use `prisma db push` como substituto do histórico versionado em produção.

11. Testes e operação

11.1 Comandos de qualidade

```
# Backend
npm run lint
npm run build
npm test -- --runInBand
npm run test:e2e -- --runInBand
npm run test:smoke
npm run test:auth-flow
```

```
# Frontend
npm run lint
npm run build
npx tsc --noEmit --noUnusedLocals --noUnusedParameters
```

11.2 Cobertura dos smoke tests

- Cadastro, autenticação e leitura de sessão.
- Catálogo consumido pelo frontend.
- Tema escuro persistido na conta.
- Carrinho, favoritos, endereço e cupom.
- Criação, pagamento demo e cancelamento de pedido.
- Baixa e restauração do estoque e contador do cupom.
- Recuperação de senha e revogação do token após logout.

11.3 Diagnóstico rápido

Sintoma	Verificações
Conexão recusada	Confirmar processos nas portas 3000 e 3001; iniciar backend antes do frontend.
Origem não permitida	Conferir URL usada no navegador e CORS_ORIGINS; não misturar host/porta indevidos.
Login retorna 401	Validar conta ativa, senha, JWT_SECRET e migrations de sessionVersion.
Produtos vazios	Verificar seed, isActive, categoria e resposta de GET /products.
Tema não persiste	Verificar PATCH /auth/settings, cookie e localStorage emarket-dark.
Checkout falha	Verificar estoque atual, endereço pertencente ao usuário, cupom e carrinho ACTIVE.

11.4 Logs e observabilidade

O NestJS registra inicialização, rotas e alertas da rotina de expiração. Erros internos não são enviados com stack trace ao cliente. Para produção, recomenda-se adicionar logs estruturados com correlation ID, métricas de latência/erro, monitoramento de banco e captura de exceções.

12. Implantação e evolução

12.1 Checklist de produção

- Definir NODE_ENV=production e secrets fora do repositório.
- Configurar HTTPS, domínio público, CORS_ORIGINS e API_URL corretos.
- Usar PostgreSQL com backup, TLS, usuário de privilégio mínimo e retenção testada.
- Executar migrations antes de liberar a nova versão.
- Manter PAYMENT_MODE e PASSWORD_RESET_MODE desabilitados até integrar provedores reais.
- Aplicar rate limiting também na borda e configurar corretamente o IP do cliente no proxy reverso.
- Executar builds, testes, npm audit e varredura de segredos no pipeline.
- Configurar monitoramento, alertas e política de rotação do JWT_SECRET.

12.2 Integrações pendentes

Capacidade	Próxima implementação recomendada
Pagamento real	Gateway com tokenização, webhooks assinados, idempotência e conciliação.
Recuperação	Provedor de e-mail, template, URL pública e fila de envio.
Login social	OAuth/OIDC com Google e Apple, state/nonce e vínculo seguro de contas.
Nota fiscal	Provedor fiscal compatível com município/estado e armazenamento do documento.
Imagens	Storage/CDN com upload administrativo, validação de MIME e transformação.
Admin frontend	Interface dedicada consumindo as rotas /admin existentes.

12.3 Riscos conhecidos

- A confirmação de pagamento atual é adequada apenas para demonstração.

- Sem provedor de e-mail, recuperação segura em produção não está completa.
- O catálogo de portfólio deve respeitar direitos de uso de marcas, textos, preços e imagens caso seja publicado.
- A rotina de expiração é interna ao processo; em múltiplas instâncias, convém mover o agendamento para fila/job distribuído.
- O CSP ainda permite estilos e scripts inline necessários à implementação atual; uma evolução pode adotar nonce por requisição.

12.4 Critérios para considerar a versão publicável

1. Todos os testes e builds aprovados em ambiente limpo.
2. Migrations aplicadas e backup restaurável validado.
3. Nenhuma credencial no Git, bundles ou logs.
4. Provedores externos configurados e testados em sandbox.
5. Revisão de autorização das rotas administrativas.
6. Teste de carga básico para login, catálogo e checkout.
7. Plano de rollback e monitoramento ativo.

Apêndice A - Regras de transição de pedido

Status atual	Próximos status aceitos
AWAITING_PAYMENT	PAID ou CANCELED; a rotina também pode marcar EXPIRED.
PAID	PREPARING ou CANCELED.
PREPARING	OUT_FOR_DELIVERY ou CANCELED.
OUT_FOR_DELIVERY	DELIVERED.
DELIVERED	Estado final.
CANCELED	Estado final.
EXPIRED	Estado final.

Apêndice B - Convenções

- Slugs: letras minúsculas, números e hífen.
- Valores monetários: Decimal(10,2) no banco e Prisma Decimal no backend.
- Peso: Decimal(10,3).
- Datas: DateTime no banco e ISO 8601 na API.
- Nomes de enum: inglês em caixa alta; mensagens ao usuário em português.
- Rotas mutáveis: POST, PATCH e DELETE; consultas: GET.

Apêndice C - Glossário

Termo	Definição
BFF	Backend for Frontend: servidor intermediário dedicado à interface.
JWT	Token assinado que representa a sessão e suas claims.

Termo	Definição
HttpOnly	Cookie inacessível ao JavaScript do navegador.
CSRF	Uso indevido da sessão por uma origem externa.
CSP	Política que restringe fontes de scripts, estilos, imagens e conexões.
Migration	Alteração versionada e reproduzível do schema do banco.
Constraint	Regra aplicada diretamente pelo banco para impedir dados inválidos.
Idempotência	Garantia de que repetir uma operação não duplica seu efeito.

Encerramento

A implementação atual estabelece uma base consistente para um e-commerce de mercado em portfólio: autenticação profissional, separação de camadas, persistência relacional, checkout transacional, controles de segurança e testes de fluxo. A evolução para produção depende principalmente da integração de serviços externos, infraestrutura observável e validação operacional em ambiente publicado.